

EcoBuck Smart Compost Monitoring System

Cross-Team Engineering Collaboration & Integration Guide

Author: Toqeer Ahmed
Role: Principal IoT Systems & Backend Architect
Date: July 8, 2026
Status: PROPOSED (Integration Alignment Stage)

1. Introduction

A successful IoT system relies on robust, well-defined boundaries between hardware sensors, edge logic, database engines, and user-facing clients. This document details the **engineering collaboration checkpoints** and **integration paths** between: 1. **Connectivity & Backend (Assignment C - Toqeer Ahmed)** 2. **Firmware & Edge Logic (Assignment B - Abdullah Ijaz & Waleed Tariq)** 3. **Mobile Application UI (Assignment D - Rabia Qaiser)**

2. Firmware (Team B) & Backend (Assignment C) Integration Points

Firmware (Team B)		Backend (Assignment C)	
- Captures raw sensor data	POST Telemetry	- Ingests & validates payload	
- Computes rolling filters		- Persists to time-series DB	
- Runs FSM (Compost State)		- Evaluates alert thresholds	
	GET Config		
- Syncs sleep/offsets		- Distributes calibration map	

2.1 Telemetry JSON Ingestion Schema Contract

- Information Needed from Firmware:
- Exact names, data types, and units of output variables (e.g. is temperature float or integer? Is time Unix timestamp or ISO string?).

- State names matching the finite state machine (`boot_self_test`, `active_composting`, etc.).

Information Firmware Needs from Backend:

- Endpoint hostname, port, and route path (e.g. `/api/v1/telemetry`).
- Specific HTTP headers (e.g., `X-Device-Token`) and expected HTTP response codes (e.g. `202 Accepted` vs `400 Bad Request`).
- **When Communication Should Happen: End of Week 1** (Milestone 1 check) before either team begins writing core client/server modules.
- **Who Should Finalize the API Contract:** Shared, with final authority given to the **Backend Owner** to prevent database schema violations.

Potential Integration Risks:

- Field naming drift (e.g., firmware team updates code to use `temperature` while backend is searching for `temperature_c`).
- Unsynchronized timestamps due to device RTC (Real-Time Clock) modules drifting on reset.

How to Avoid Integration Issues:

- Generate a single shared JSON Schema contract stored in the root workspace.
- Incorporate automatic Pydantic schema validation inside the ingestion middleware to fail early with clear warnings.
- Backend automatically overrides timestamps if they drift from server time by more than 10 minutes, generating a sync warning.

2.2 Device Authentication & Cryptographic Handshake

Information Needed from Firmware:

- How security tokens are written to flash memory (e.g. non-volatile storage partition limits).
- Support for standard hashing libraries on the ESP32.

Information Firmware Needs from Backend:

- The hash algorithm requirements (HMAC-SHA256) and the raw authentication token generated at device creation.
- **When Communication Should Happen: Week 2** (During authentication setup).
- **Who Should Finalize the API Contract: Backend Owner** (Security Policy Enforcer).

Potential Integration Risks:

- Tokens accidentally logged in clear text on firmware debug consoles or server output monitors.

- Authentication failures due to character encoding mismatches during payload hashing.

How to Avoid Integration Issues:

- Define token standards using base64-encoded URL-safe strings.
- Write integration unit tests validating authentication logic using mock clients *prior* to hardware bench testing.

2.3 Configuration Sync & Threshold Distribution

Information Needed from Firmware:

- Calibration offset values matching physical sensor tests.
- The maximum reading frequency capabilities and battery-saving sleep schedules.

Information Firmware Needs from Backend:

- The configuration query path (`GET /api/v1/devices/config`) and the structure of the returned JSON profile.
- **When Communication Should Happen: Week 3** (Database schema setup).
- **Who Should Finalize the API Contract: Backend Owner.**

Potential Integration Risks:

- The device fails to connect and hangs indefinitely, preventing it from reading sensors.
- Configuration updates returning floats that cause division-by-zero or value errors in the firmware logic.

How to Avoid Integration Issues:

- Firmware must implement fallback defaults hardcoded locally inside `config.py` in case connection attempts fail.
- Limit range settings on the backend validation layer to prevent sending out-of-bounds configuration constants.

2.4 Compost Batch Lifecycle Reset

Information Needed from Firmware:

- How the FSM resets its internal stability duration timer and cycle age metric upon validation.

Information Firmware Needs from Backend:

- How the validation command is received by the device (e.g., a flag returned in config payload: `"reset_cycle": true`).
- **When Communication Should Happen: Week 6** (End-to-End integration).
- **Who Should Finalize the API Contract:** Shared.

Potential Integration Risks:

- Desynchronization where the backend registry resets the cycle age to Day 0, but the device continues tracking the old timeline because it missed the config sync update.

How to Avoid Integration Issues:

- Introduce a unique `cycle_id` UUID generated by the backend database.
- The firmware stores this `cycle_id` locally. If it detects a change in `cycle_id` from the config response, it automatically triggers an internal state reset.

3. Mobile Application (Assignment D) & Backend Integration Points

Mobile App (Assign D)		Backend (Assignment C)	
■ - Displays Dashboard charts	■ GET Dashboard	■ - Serves cached status lists	■
■ - User logins & registrations	■	■ - Runs time-series query APIs	■
■ - Receives push alerts (FCM)	■	■ - Connects Firebase Messaging	■
■	■ POST Validate	■	■
■ - Triggers manual confirm	■	■ - Records audit validations	■

3.1 Real-Time Dashboard Status (Latest Caching)

Information Needed from Mobile:

- The complete list of widgets and variables required for the main display dashboard (e.g. does it need battery percentage, FSM states, sensor status, or raw metrics?).

Information Mobile Needs from Backend:

- The cache document structure `/devices/{id}/latest` and listener protocols.
- **When Communication Should Happen: Week 3** (Database setup).
- **Who Should Finalize the API Contract:** Shared.

Potential Integration Risks:

- Mobile app pulling data via constant HTTP polling loops, resulting in heavy database read cost charges.

How to Avoid Integration Issues:

- Expose a single status document read. Enforce real-time listeners (Firebase `onSnapshot`) or WebSockets, avoiding loop polling.

3.2 Historical Graph Data (Trends)

Information Needed from Mobile:

- Chart timeline resolutions (e.g., weekly, monthly view requirements) and chart framework limitations.

Information Mobile Needs from Backend:

- REST query parameters support (e.g. `resolution=hourly` or `resolution=daily`) on route `/devices/{id}/history`.
- **When Communication Should Happen: Week 3.**
- **Who Should Finalize the API Contract: Backend Owner.**

Potential Integration Risks:

- Query requests timeout when fetching massive ranges of raw time-series data.

How to Avoid Integration Issues:

- Implement query size caps on the backend (e.g. limit queries to max 90 days).
- Return downsampled averages instead of high-frequency raw points for wide search ranges.

3.3 User Authentication & Authorization Scopes

Information Needed from Mobile:

- User identity requirements (social login tokens vs email logins).

Information Mobile Needs from Backend:

- Authentication schemas (`POST /auth/register`, `POST /auth/login`), JWT payload validation structures, and token refresh protocols.
- **When Communication Should Happen: Week 2.**

- **Who Should Finalize the API Contract: Backend Owner.**

Potential Integration Risks:

- Mobile client fails to store JWT securely, or login state resets prematurely on network drops.

How to Avoid Integration Issues:

- The backend provides Swagger/OpenAPI documentation mapping refresh token exchanges.
- Mobile developers must persist tokens in secure device keychains.

3.4 Push Notifications Integration (Firebase Cloud Messaging)

Information Needed from Mobile:

- The registration token generated by the mobile OS push engine.

Information Mobile Needs from Backend:

- The notification payload formats (title, body description, routing keys to open the alerts page).

- **When Communication Should Happen: Week 4** (Alert engine setup).
- **Who Should Finalize the API Contract: Shared.**

Potential Integration Risks:

- Push tokens expiring or breaking without notifying the server database.

How to Avoid Integration Issues:

- Expose an endpoint `/users/fcm-token` that the mobile client calls upon every app startup to ensure tokens are always fresh.

3.5 Manual Validation Action Flow

Information Needed from Mobile:

- Notes and inspection data captured by the user during physical confirmation checks.

Information Mobile Needs from Backend:

- The validation route `/devices/{id}/validate` and custom HTTP error code meanings (e.g. error 400 when user tries to validate a cycle that hasn't completed).

- **When Communication Should Happen: Week 6** (Integration testing).
- **Who Should Finalize the API Contract: Shared.**

Potential Integration Risks:

- Multiple family members attempt to validate the same compost batch concurrently, causing status overlap issues.

How to Avoid Integration Issues:

- The validation controller must execute transactional queries, checking if status is strictly `ready_candidate` before proceeding with update commands.